

CANOPY: finding tree crowns, and saying how sure it is

Soumyadeep Dey · solo submission · repo: github.com/SoumyaEXE/canopy

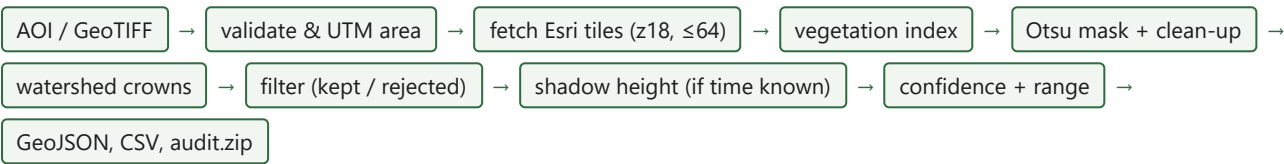
What I built and why

CANOPY takes an area (drawn on a map, or a GeoJSON, KML, KMZ or GeoTIFF file) and outlines the tree crowns it can see in satellite imagery. It reports canopy cover, a tree count given as a **range**, a confidence score for every crown, shadow-based heights only when the maths is actually possible, and a zip file that records every parameter behind every number.

Early on I made a call: I would rather ship a tool that is honest about what it doesn't know than one that shows neat numbers it can't back up. Most of the design follows from that. Counts are ranges. Rejected regions get their own map layer instead of vanishing. Height switches off when the capture time is unknown. There's no carbon figure at all.

Architecture and workflow

Frontend: React 19 + TypeScript + Vite, Tailwind v4 with BoardUI components, MapLibre GL for the map. **Backend:** FastAPI (Python 3.12) running a background job per analysis. The frontend polls for stage progress. Projects and run history live in SQLite, with output files on disk.



- Input.** Broken geometry gets repaired. Multi-part polygons are rejected, not quietly merged. Area is measured in the local UTM zone and capped at 1 km². A GeoTIFF with no coordinate system is refused, because I won't guess where it is.
- Imagery.** Esri World Imagery tiles at zoom 18 are stitched and cropped. If a tile still fails after retries, the whole job fails. I didn't want a blank patch to read as "no trees here".
- Canopy mask.** Excess Green (2g-r-b) by default. VARI is optional, and NDVI is used if a near-infrared band exists. Otsu picks the threshold, and Sarle's bimodality coefficient (cutoff 5/9) flags when that split is forced rather than real.
- Crowns.** A smoothed distance transform seeds a marker-controlled watershed, which splits touching canopy. Regions that are too small, larger than 400 m², or too irregular are rejected but still counted and drawn.
- Height.** The NOAA sun-position algorithm plus three shadow rays per crown gives `height = shadow × tan(sun elevation)`. It only runs when the capture time is known.
- Confidence.** A weighted mix of shape (0.30), size vs. its 8 nearest neighbours (0.25), separation (0.25) and shadow plausibility (0.20). Range = high+medium up to high+medium+2×low, because low-confidence blobs are often two merged trees.
- Validation mode.** You mark trees yourself in a patch (at least 15 clicks, with detections hidden). It returns precision, recall, F1 and a correction factor. It can also fit a tiny NumPy-only logistic regression for a locally calibrated count.

Key technical decisions

Decision	Why
Classical computer vision, no deep learning model	No labelled data for this task, a short deadline, and free CPU hosting. A classical pipeline is explainable parameter by parameter, and it's deterministic. The trade-off is lower accuracy in dense forest.
Deterministic outputs	Sorted JSON keys, fixed zip timestamps, rounded coordinates. The same input gives a byte-identical <code>crowns.geojson</code> , and a test checks it.
Ranges and confidence instead of one count	A single number hides the main failure (merged crowns). The range makes that uncertainty visible.
Refuse instead of guess	No default capture time, no assumed CRS, no carbon/biomass conversion. Each of those would produce numbers that look precise and aren't.
Checked the scale against a real object	I measured Victoria Memorial, Kolkata, from the mosaic: area came within ~2% of published figures, and each axis within 10%.

What worked

- **Open woodland.** On the sample area (Monfragüe dehesa, Spain: scattered holm oaks on dry grass) it finds 532 crowns, range 479–585, with 20.8% cover. That matches what I see when I look at the imagery.
- **Threshold-quality check.** My first rule (between-class / total variance) couldn't tell a real split from a forced one. It read ~0.64 even on a single Gaussian. Sarle's coefficient fixed that: a near-uniform grass scene (the Maidan, Kolkata) correctly reads "low", and the sample reads "high".
- **Testing.** 36 backend unit tests pass, including determinism. An end-to-end API script passes 19/19 checks across drawn AOI, KML, KMZ, GeoJSON, GeoTIFF, bad-CRS rejection, oversize errors, validation and audit bundle contents. The frontend builds cleanly, and I ran through the main flows in headless Chromium, including a 390 px phone layout.
- **Audit trail.** Every run keeps its inputs, parameters, manifest and outputs, so you can reopen and compare old runs.

What didn't work (or only partly)

- **Lawns and crops.** My first test area in Kolkata showed ExG marks irrigated lawns as canopy. RGB alone can't separate a lawn from a tree canopy, so I moved the sample to a dry-grass site. That's a workaround, not a fix.
- **Valley-depth threshold confidence** was unstable, and I dropped it.
- **Heights in the live demo.** Esri basemap tiles have no reliable capture time, so height is off for nearly every drawn area. The feature really only works for GeoTIFFs with a timestamp, or if the user types the time in.
- **Deployment was messy.** I tried Render (free tier sleeps, no persistent disk) and a Hugging Face Space (needed a dummy Gradio wrapper), then ended up on my own VM behind nginx. The Docker image was never test-built.

Known limitations (honest list)

- **No real accuracy numbers yet.** Validation mode works, but I haven't done a proper hand-marked validation on an open and a dense patch. "Matches visual inspection" is my eyeballing, not a measured precision/recall. This is the biggest gap.
- **Dense forest undercounts.** In closed canopy, touching crowns merge into one. The count range tries to show this, but the "2× low" upper bound is a rule of thumb, not something I derived statistically.
- **Confidence weights are hand-picked.** 0.30/0.25/0.25/0.20 and the high/medium cutoffs aren't calibrated against ground truth. "High confidence" means "looks like a clean crown", not "is 90% likely to be a tree".
- **"Tree" really means "green blob of tree size".** Hedges, shrubs, green roofs and crops can pass. There's no species, age, health, understory, or carbon estimate, and that's on purpose.
- **Heights** assume flat ground and a clear shadow. They're also biased low by roughly crown radius × tan(sun elevation), since the shadow is measured from the crown edge.
- **Imagery.** Tile capture dates are unknown and can vary across a mosaic. I haven't fully checked Esri's terms for a public demo.
- **Scale and ops.** Area is capped at 1 km² / 64 tiles. Jobs live in memory, so a restart loses them. It has to run as a single worker. Projects need a persistent disk or they reset on redeploy. The VM service currently allows CORS from any origin, which was a shortcut to get the demo working and should be locked down. Tile-cache hits aren't recorded in the manifest.
- **Smaller shortcuts.** The upload component shows a simulated progress ring before the real upload starts. The calibration model is tiny, only valid for the patch it was trained on, and needs at least 4 matched and 4 unmatched examples. The wordmark font's web licence isn't confirmed yet.

What I'd do next

First, a proper hand-marked validation on two contrasting patches, with the numbers published in the app. Then calibrate the confidence weights against those labels, add a job store (Redis) so restarts don't lose work, and try a small pretrained crown detector (e.g. DeepForest) as an optional second opinion next to the classical pipeline.

Built with substantial help from AI coding assistants (Claude Code, Cursor). I made the design decisions, checked the scale and threshold behaviour by hand, and reviewed the output. Full method: [docs/METHOD.md](#); build notes: [NOTES.md](#).